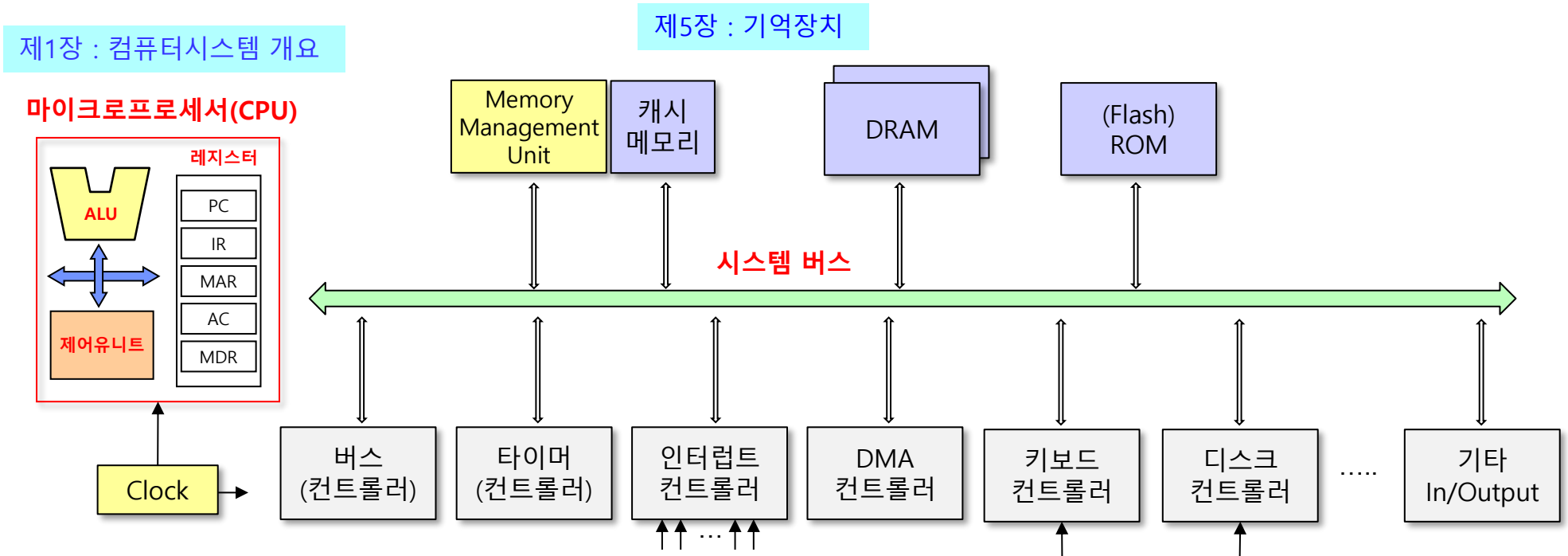
컴퓨터구조

Presented by
오 세 춘

2-2장 CPU의 구조와 기능(명령어 파이프라이닝)

교재 주요 구성

■ 컴퓨터 시스템의 구성 – 교재 및 강의 구성 내용



배울 내용

■ 목차

- ✓ 2.1 CPU의 기본 구조
- ✓ 2.2 명령어 실행

} 약 1주간 학습

- ✓ 2.3 명령어 파이프라이닝 — 약 1주간 학습
  - 1) 개요
  - 2) n-단계 파이프라이닝
  - 3) 효율저하 및 개선책
  - 4) 슈퍼스칼라/멀티-코어 → 명령어 수준 병렬성 포함
  - 5) 산술 파이프라인
- ✓ 2.4 명령어 세트 — 약 1주간 학습

2.3 명령어 파이프라이닝

p82~p91

■ 명령어 파이프라이닝(instruction pipelining)

✓ CPU의 처리 속도를 높이기 위하여 CPU 내부의 하드웨어를 여러 단계로 나누어 병렬로 동시에 처리하는 기술

➢ 근래에 사용되는 거의 모든 CPU에서 사용하는 프로세서의 성능 향상을 위한 기법

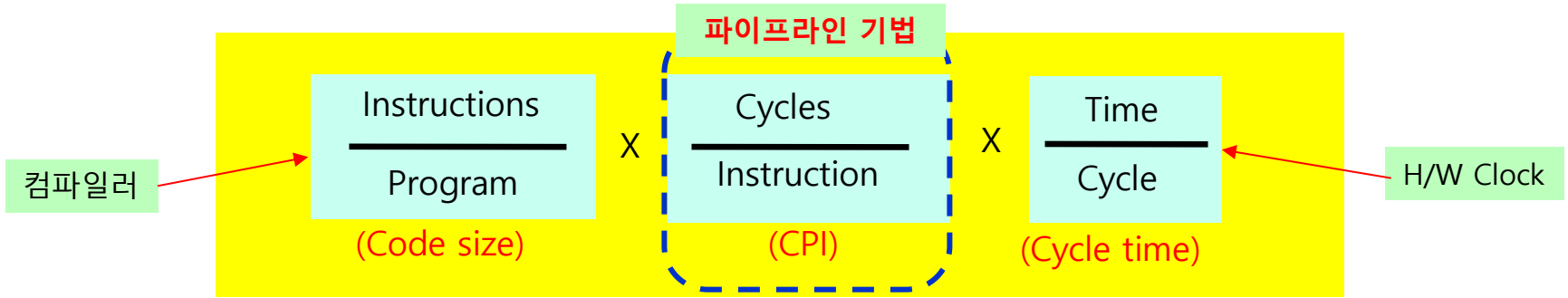
✓ 컴퓨터에서의 처리속도 향상을 위한 다양한 기법들

1) 코드 사이즈의 최적화 → 컴파일러

2) CPI(Cycle Per Instruction)의 최소화 → H/W 설계

3) System Clock Cycle의 고속화 → H/W 설계

파이프라인의 종류는 명령어 파이프라인, 산술 파이프라인이 있다. 특별히 명시하지 않으면 **명령어 파이프라인**을 의미



✓ 파이프라인은 프로세서의 명령어 처리 시간을 줄이는 기법 (파이프라인 기법 = 파이프라이닝)

➢ 즉 처리시간에서 **한 명령어당 평균 처리 Cycle 수(CPI)를 줄이는 방법**

➢ CPU 내에 H/W로 구성 → 대부분의 RISC에 적용

2.3 명령어 파이프라이닝

p82~p91

■ 2-단계 명령어 파이프라인(two-stage instruction pipeline)

✓ 명령어를 실행하는 하드웨어를 ①인출 단계(fetch stage)와 ②실행 단계(execute stage)라는 두 개의 독립적인 **파이프라인** 모듈로 분리

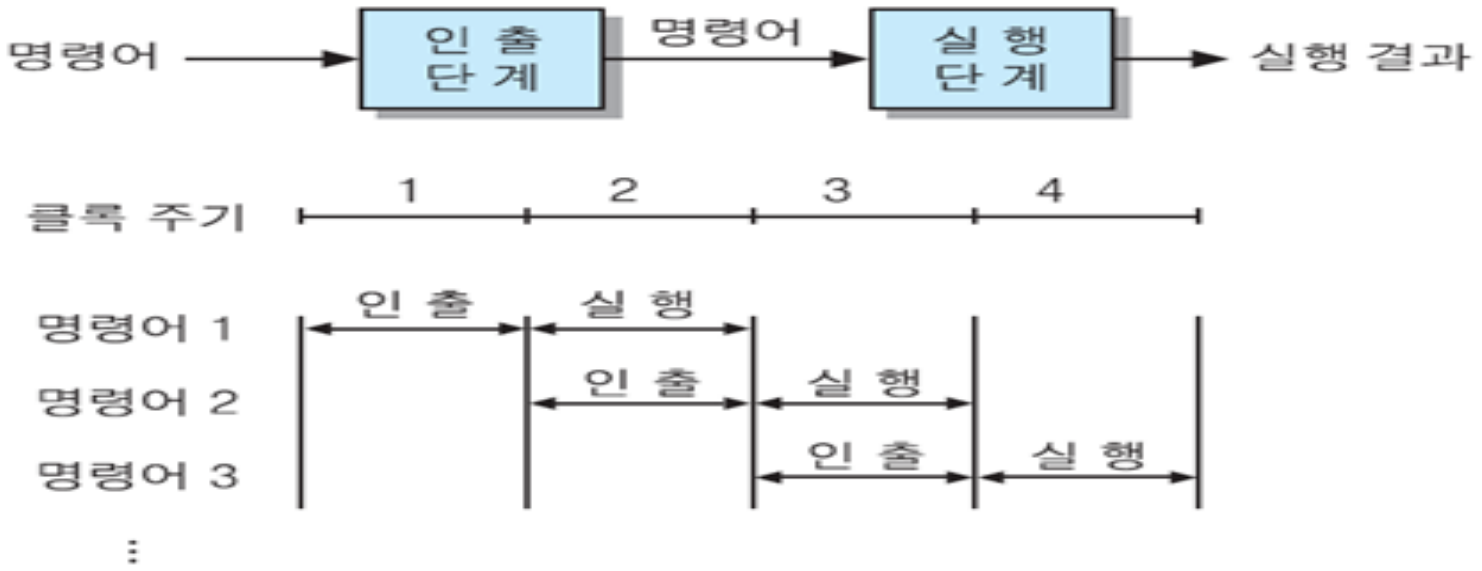
→ **2-statge**(스테이지)

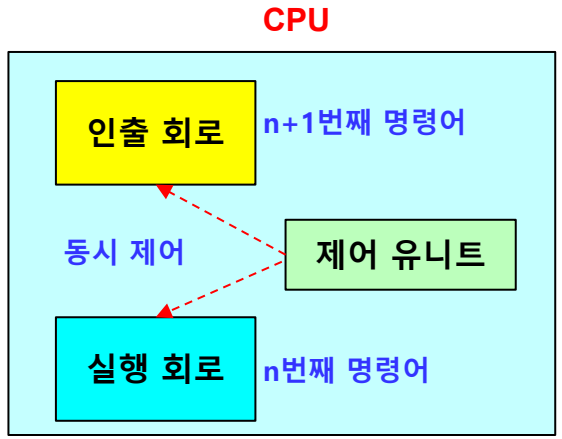
✓ 두 단계들의 실행시간을 동일하게 설계하면,

① 첫 번째 클록 주기에서는 인출 단계(회로)가 첫 번째 명령어를 인출

② 두 번째 클록 주기에서는 인출된 첫 번째 명령어가 실행 단계로 보내져서(파이프라인) 실행되며, 그와 동시에 인출 단계는 두 번째 명령어를 인출

파이프라인의 어원
유체를 파이프를 연결해 한 방향으로 지속적으로 보낼 때 사용하는 파이프들의 묶음을 '파이프라인' 이라고 함





2.3 명령어 파이프라이닝

p82~p91

■ 2-단계 명령어 파이프라인 – 계속

- ✓ 2-단계 파이프라인을 이용하면 명령어 처리 속도가 두 배로 향상
  - ➢ 인출된 명령어가 실행 되면서 **동시**에 다음 명령어가 인출된다
    - ❖ 장기적으로 보면 2개의 명령어가 동시에 실행되는 효과
- ✓ 문제점
  - ➢ 두 단계의 처리 시간이 동일하지 않으면 두 배의 속도 향상을 얻지 못함 → 파이프라인 효율 저하
- ✓ 해결책
  - ➢ 파이프라인 단계(stage)의 수를 증가(기능 단위를 쪼개어)시켜 각 단계의 처리 시간을 같게 함

CISC(Complex Instruction Set Computer)에서는 **명령어들의 길이가 가변적**이라 인출단계에서 **인출에 필요한 시간이 일정하지 않으며**, 또한 제어 유니트(장치)도 **마이크로 프로그램 제어방식**을 사용하기 때문에 명령어들의 **실행 속도도 일정하지 않기 때문**에 **명령어 파이프라인의 적용이 매우 비효율적**이다. 따라서 근래의 CPU는 대부분 RISC 방식을 기본으로 설계한다.

2.3 명령어 파이프라이닝

p82~p91

■ 2-단계 명령어 파이프라인 – 계속

✓ AVR 8비트 MCU는 4단계 명령어 사이클을 가지며 2-단계 명령어 파이프 라인을 사용

➢ 4단계 명령어 사이클

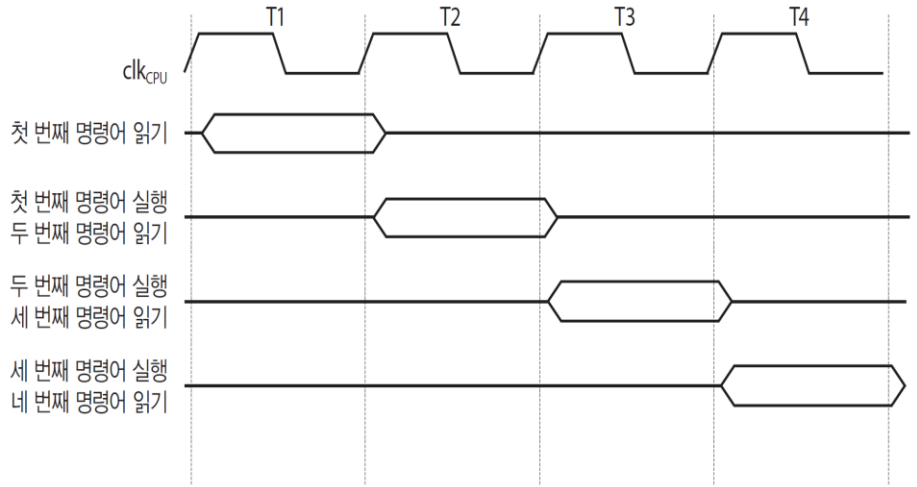
❖ ①명령어 인출 → ②레지스터/피연산자 읽기 → ③ALU 내의 연산 → ④결과 쓰기

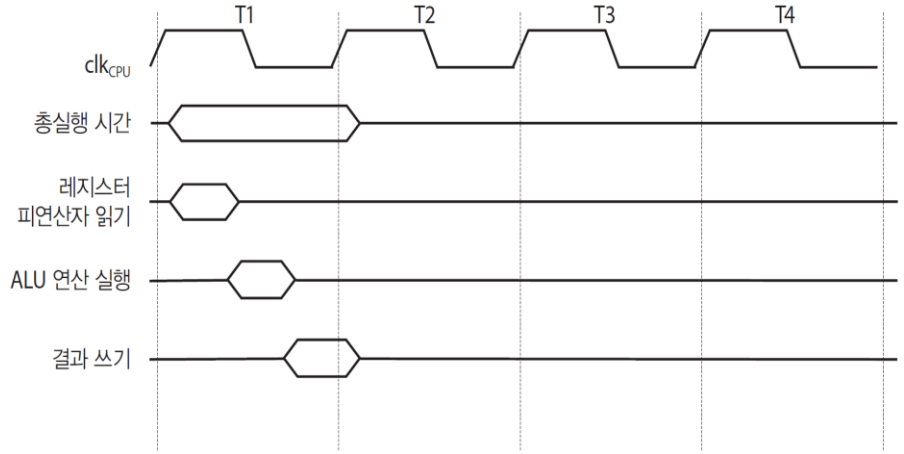
➢ 2-단계 파이프라인 → 1 CPI(Cycles Per Instruction)

❖ 명령어 읽기(①)와 명령어 실행(②~④)

➢ 실행 단계는 3단계의 세부 단계로 나뉘어 진행

❖ ②레지스터/피연산자 읽기 → ③ALU 내의 연산 → ④결과 쓰기





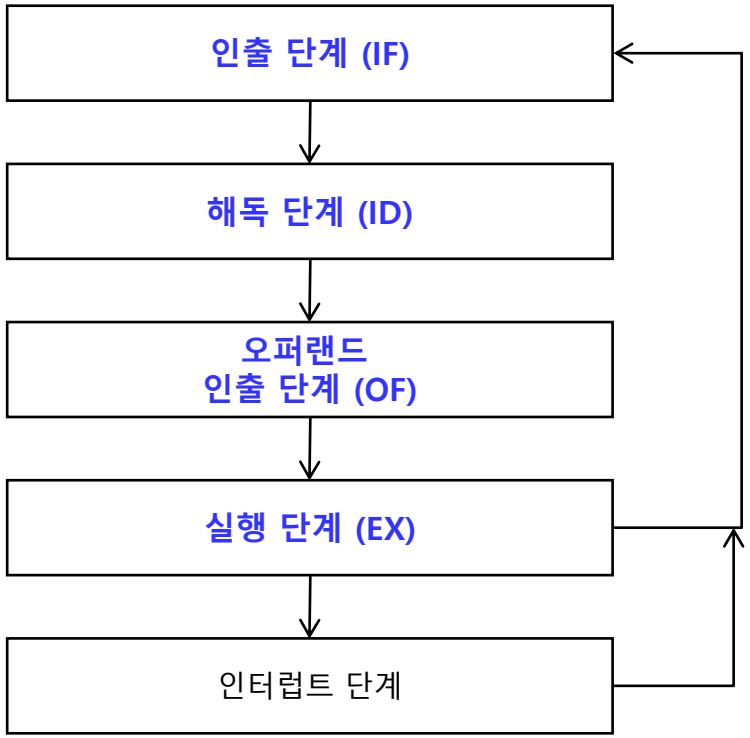
2.3 명령어 파이프라이닝

p82~p91

■ 4-단계 명령어 파이프라인

파이프라인 단계의 수를 늘릴수록 속도 향상이 더욱 커짐

- ✓ 1단계: 명령어 인출(IF) 단계
  - ➢ Instruction Fetch
  - ➢ 다음 명령어를 기억장치로부터 인출
- ✓ 2단계: 명령어 해독(ID) 단계
  - ➢ Instruction Decode
  - ➢ 해독기(decoder)를 이용하여 명령어(연산자)를 해석
- ✓ 3단계: 오퍼랜드 인출(OF) 단계
  - ➢ Operand Fetch
  - ➢ 필요시 기억장치로부터 오퍼랜드를 인출
  - ➢ 인출할 오퍼랜드가 없으면 아무것도 안 하면서 시간을 소비
- ✓ 4단계: 실행(EX) 단계
  - ➢ Execution
  - ➢ 지정된 연산을 수행

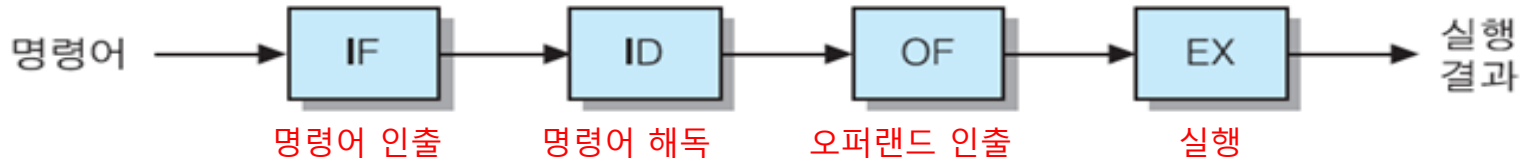


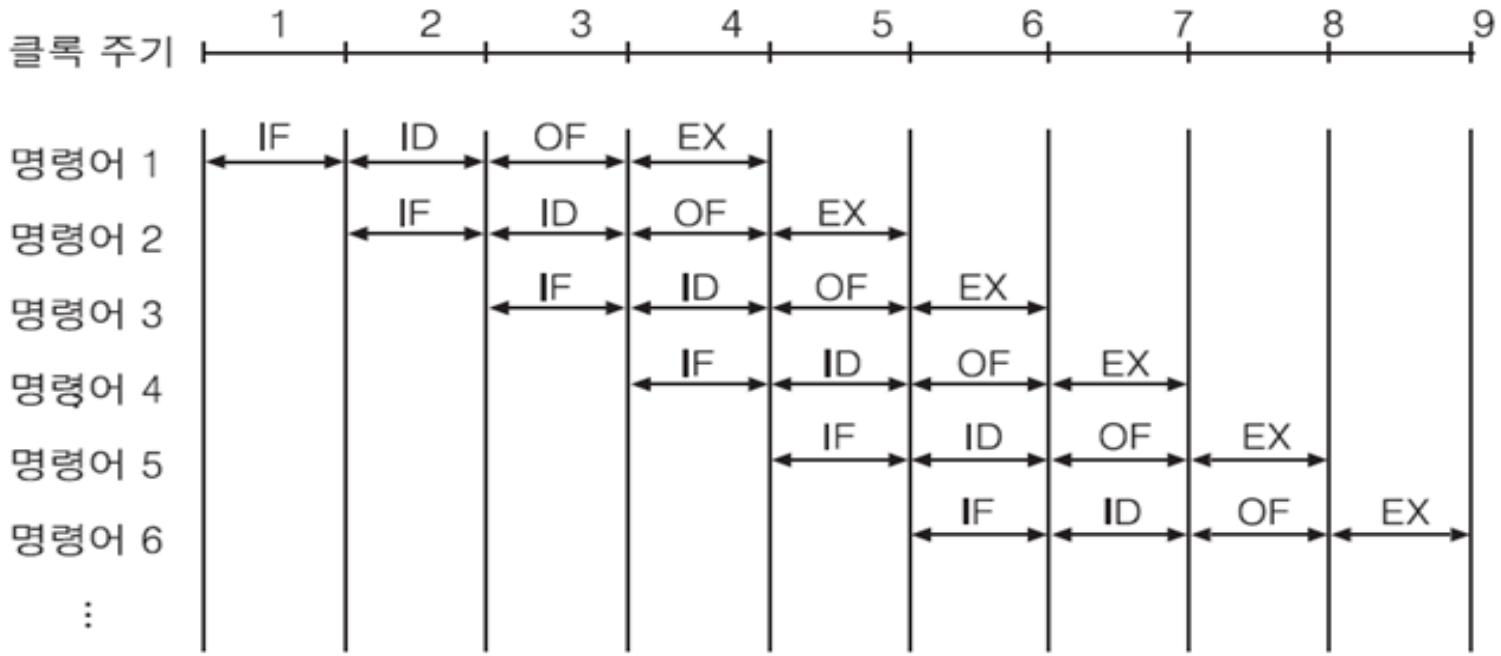
< 4-단계 명령어 사이클 >

2.3 명령어 파이프라이닝

p82~p91

■ 4-단계 명령어 파이프라인과 시간 흐름도





< CPU 제어 유니트 >



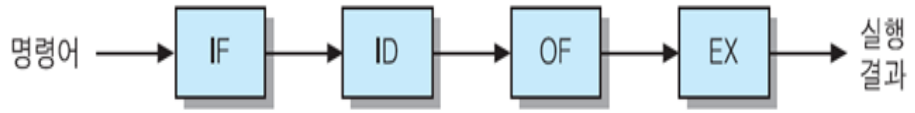
2.3 명령어 파이프라이닝

p82~p91

■ 파이프라인에 의한 전체 명령어 실행 시간

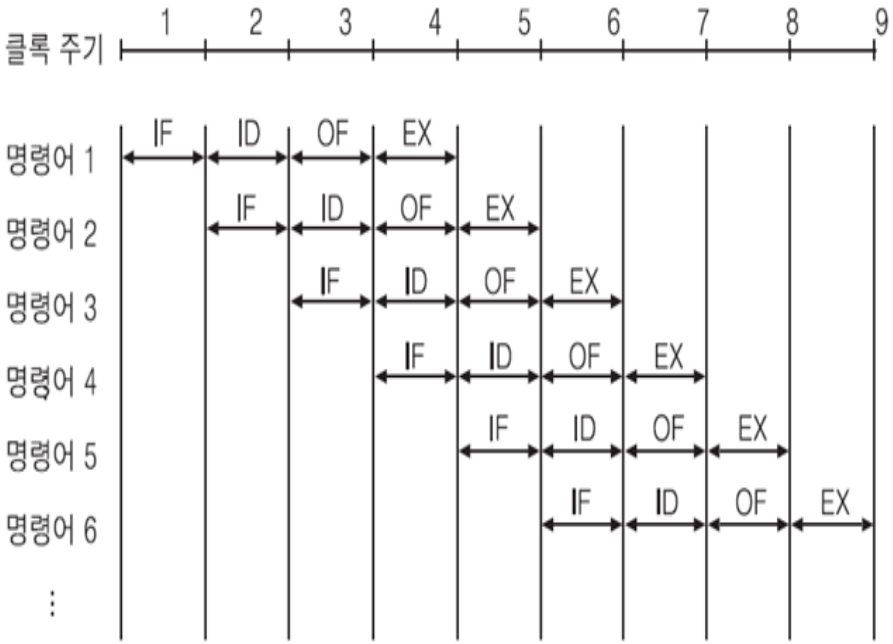
✓ 조건

- ➢ 파이프라인 단계 수 = k
- ➢ 실행할 명령어들의 수 = N
- ➢ 각 파이프라인 단계가 한 클록씩 걸린다고 가정한다면



✓ 파이프라인 되지 않은 경우의 N 개의 명령어들을 실행 시간 (T_1)

$$T_1 = k \times N$$



✓ 파이프라인에 의한 전체 명령어 실행 시간(T_k)

$$T_k = k + (N - 1)$$

- ➢ 첫 번째 명령어를 실행하는데 k 주기가 걸리고,
- ➢ 나머지 $(N - 1)$ 개의 명령어들은 각각 한 클록씩 만 소요

2.3 명령어 파이프라이닝

p82~p91

■ 파이프라인에 의한 속도 향상(speedup)

$$S_p = \frac{T_1}{T_k} = \frac{k \times N}{k + (N - 1)}$$

파이프라인 단계 수 = k

실행할 명령어들의 수 = N

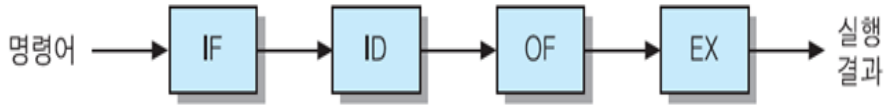
2.3 명령어 파이프라이닝

p82~p91

■ [예제 2-1] 파이프라인에 의한 속도 향상

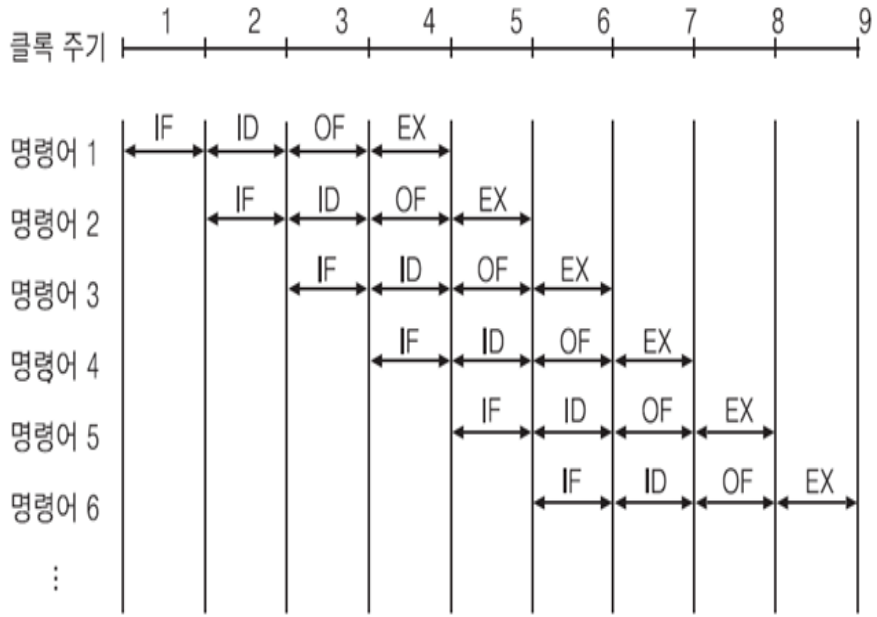
✓ 문제

- 파이프라인 단계 수 = 4, 파이프라인 클록 = 1GHz (각 단계에서의 소요시간 = 1/1GHz = 1/(1x10^9)Hz = 10^{-9}초 = 1ns)일 때,
- 10개의 명령어를 실행하는 경우의 속도향상은?



✓ 풀이

- 첫 번째 명령어 실행에 걸리는 시간 = 4ns
- 다음부터는 1ns 마다 한 개씩의 명령어 실행 완료
- 10개의 명령어 실행 시간 = 4 + (10 - 1) = 13ns
- 따라서 속도향상(speedup: Sp) = (10 × 4) / 13 ≒ 3.08 배



✓ 참고

- CPU 가 실행하는 명령어 수가 N 일 경우
- N = 100 일 때, Sp = 400/103 = 3.88
- N = 1000 일 때, Sp = 4000/1003 = 3.988
- N = 10000 일 때, Sp = 40000/10003 = 3.9988
- $N \rightarrow \infty$ 일 때는 $Sp \rightarrow$ 4 (이론적 속도향상)

2.3 명령어 파이프라이닝

p82~p91

■ 파이프라인의 효율 저하 요인들

✓ 단계를 늘려도 성능은 무한정 향상되지 않음 → 이론적인 성능만큼 개선되지 않음

✓ **파이프라인의 제약조건**

1) 모든 명령어들이 파이프라인 단계들을 모두 거치지는 않는다

❖ 어떤 명령어에서는 오퍼랜드를 인출할 필요가 없지만, 파이프라인의 하드웨어를 단순화시키기 위해서는 모든 명령어가 네 단계들을 모두 통과하도록 해야 한다 → 아무것도 안 하면서 주어진 시간을 소비

2) 파이프라인의 클록은 처리 시간이 가장 오래 걸리는 단계를 기준으로 결정된다

3) 시스템 자원의 사용에 충돌이 없어야 함

❖ 코어 내의 동일 레지스터, ALU 등이 동시에 다른 일을 못함

❖ IF 단계와 OF 단계가 동시에 기억장치를 액세스하는 경우에, 기억장치 충돌(memory conflict)이 일어나면 지연이 발생한다

4) 명령어는 순차적으로 실행해야 이상적 → 명령어 종속성

❖ 무조건 분기(인터럽트, 분기 명령어 등) 발생시에는 미리 인출하여 처리하던 명령어들이 무효화로 인한 지연

❖ 조건 분기(conditional branch) 명령어가 실행되면, 미리 인출하여 처리하던 명령어들이 무효화된다

5) 추가: 명령어간 상호 의존성이 없어야 함 → 데이터 종속성

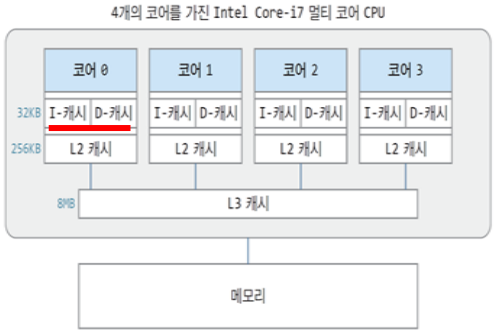
❖ 현재의 명령어가 이전 명령어(아직 결과가 나오지 않은)의 값을 참고할 경우

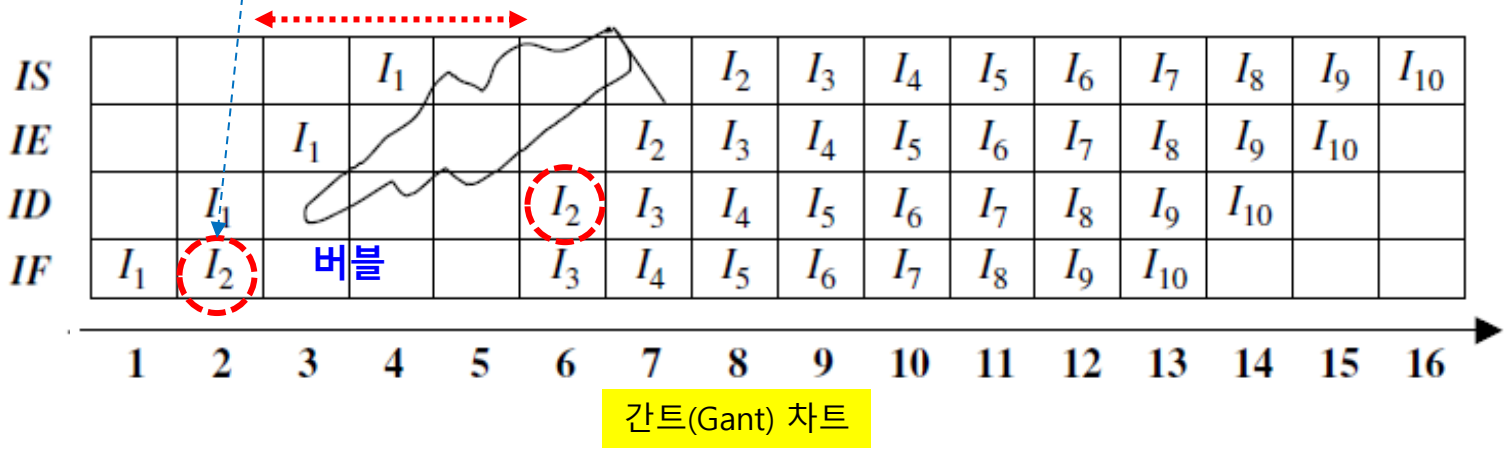
2.3 명령어 파이프라이닝

■ 파이프라인 스톨 (Stall)

- ✓ 파이프라인의 기능이 잠시 꺼지는 상태를 '파이프라인 스톨'이라 하며 파이프라인이 꺼져서 파이프라인 스테이지가 비게 되는 사이클을 버블(Bubble=거품)이라 한다.(아래 그림의 빈칸들)
- ✓ 예를 들어 I_2 명령어 인출 시 캐시가 실패되어 주메모리에서 명령어를 인출해야 할 경우에 추가로 3T(사이클)이 필요하다면
  - ➢ I_2에서의 캐시 실패 후 I_2를 인출하는데 추가적인 3T가 더 필요하다면
  - ➢ 이 경우 나머지 단계들을 Idle 상태로 되면서 멈추는데 이런 상태가 '파이프라인 스톨'이다
- ✓ 이러한 스톨 상태를 만드는 원인들을 **파이프라인 해저드**라 한다.

CPU 외부 메모리 접근의 경우 속도가 느려 파이프라인 스톨이 될 가능성이 높기 때문에 요즘 CPU들은 CPU 내부에 하버드 구조를 사용한 캐시메모리를 사용





간트(Gant) 차트

IF: Instruction Fetch (인출)
ID: Instruction Decode (해독)
IE: Instruction Execute (실행)
IS: Instruction Store (저장)

2.3 명령어 파이프라이닝

■ 파이프라인 해저드 (Pipeline Hazard)

해저드 = 위험(요소)

- ✓ 파이프라인의 속도를 느리게 만드는 원인들을 말함
- ✓ 즉 다음 명령어가 다음 클록 사이클에 인출 또는 실행될 수 없는 상황(스톨)을 만드는 원인

■ 종속성 (dependency)

- ✓ 한 명령어의 실행 상태가 다른 명령어의 실행에 영향을 미치는 상태를 말함

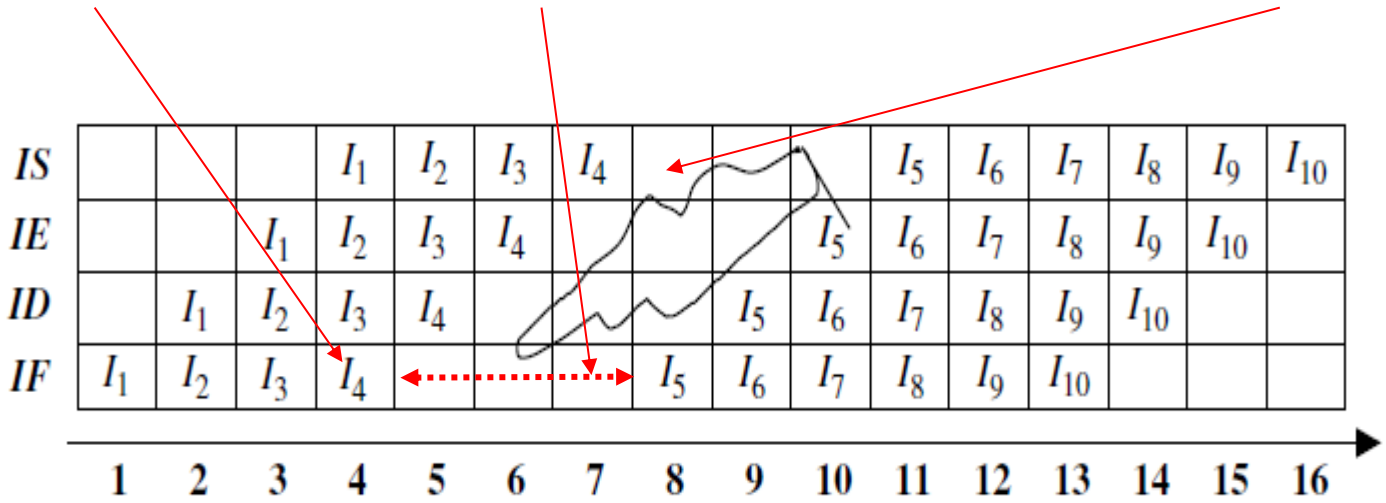
■ 파이프라인 해저드의 종류

유형	세부 설명	해결책
구조 해저드	하드웨어 (구조)가 여러 명령의 수행을 지원하지 않기 때문에 발생 → 자원 충돌	자원 분리, 파이프라인용 명령어 재설계 등
명령어 해저드 (제어 해저드)	분기 명령에 의해서 발생 → **명령어 종속성**	분기예측, 지연(stall) 등
데이터 해저드	현 명령이 현재 파이프라인에서 수행 중인 이전 명령의 결과(데이터)에 종속되는 경우 발생 → **데이터 종속성**	데이터 포워딩, 지연(stall) 등

2.3 명령어 파이프라이닝

■ 명령어 종속성

- ✓ 새로운 명령어를 **인출(Fetch) 할 때** 앞 단의 결과에 따라 **인출 여부가 달라지는 경우**를 말함 → Control(제어) Hazard라고도 부름
- ✓ 명령어 종속성의 예
  - ➢ 무조건 분기 명령 → Jump, Call 명령어나 인터럽트 요청
  - ➢ 조건부 분기 명령 → 앞 단의 명령어가 "branch if negative : 연산결과가 음수이면 jump" 일 경우 연산 결과가 완전히 저장되고 사용될 수 있을 때까지 새로운 명령어의 인출 여부는 확인할 수 없다.
- ✓ 위의 조건부 분기 명령의 경유를 간트 차트로 설명하면 다음과 같다
  - ➢ I_4명령어가 조건부 분기 명령일 경우 그 결과가 저장될 때까지 새로운 명령어의 인출을 안하고 명령어 파이프라인은 스톨 된다.



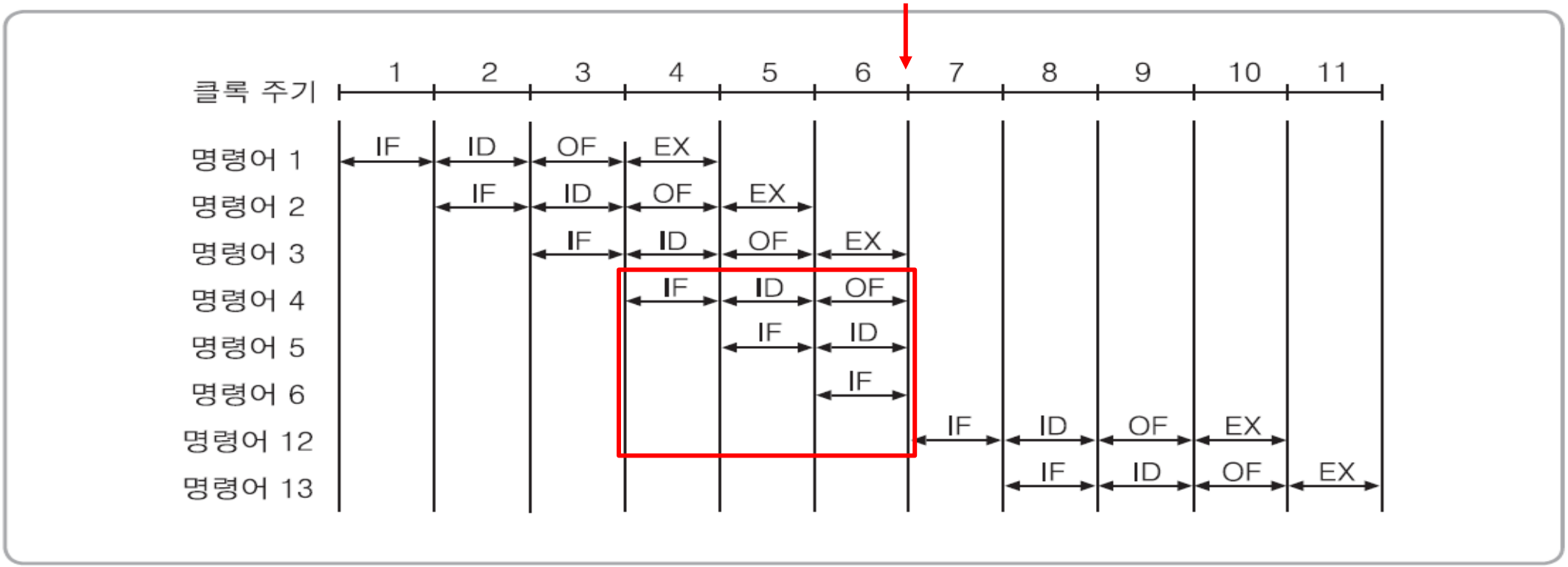
2.3 명령어 파이프라이닝

p82~p91

■ 조건 분기(명령어 종속성)가 존재하는 경우의 시간 흐름도

✓ [예] 명령어 3: **JZ 12** ; jump (if zero) to address 12(상태 레지스터의 제로 플래그가 1이면 12번지로 점프)

➢ 만약 12번지로 조건 분기를 하게 되면 붉은 박스 부분의 미리 처리된 내용들은 의미가 없어짐



2.3 명령어 파이프라이닝

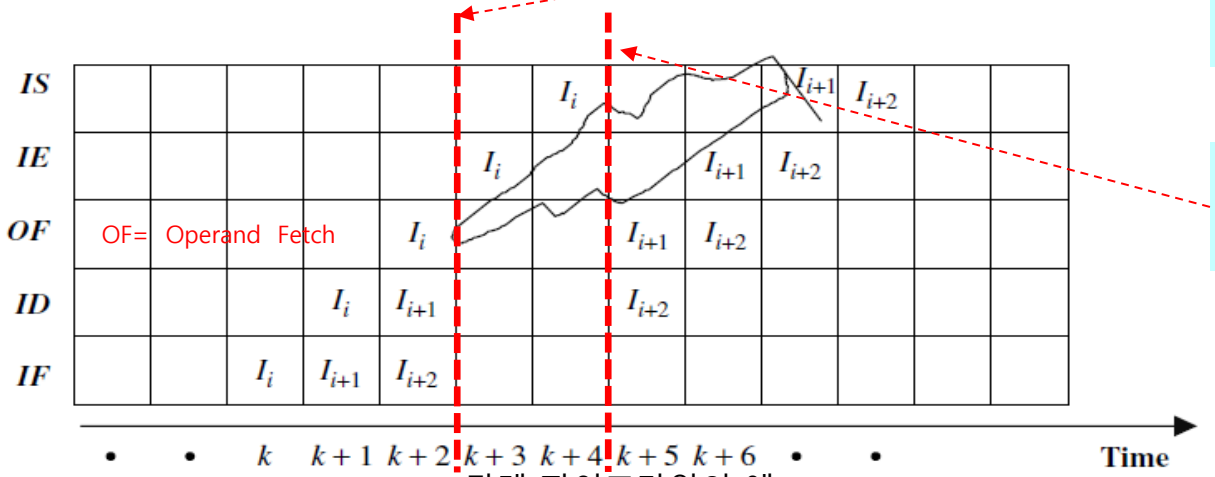
■ 데이터 종속성

- ✓ 현재 명령어의 소스 오퍼랜드가 이전의 명령어의 수행 결과 값과 연관이 있을 때를 데이터 종속성이 있다고 한다 → **인출은 되었으나 실행이 안됨** (인출 자체가 안되는 것은 명령어 종속성)
- ✓ 즉 **이전 명령어의 수행 결과가 저장되기 전에 현재의 명령어에서 이를 사용하려고 하는 경우**
- ✓ 데이터 종속성의 예

① ADD R1, R2, **R3** ; R3 = R1 + R2

② SL **R3** ; R3 = SL(R3), R3를 1 비트 좌측 이동

③ SUB R5, R6, R4 ; R4 = R5 – R6



< 5-단계 파이프라인의 예 >

①ADD 명령어가 오퍼랜드 읽어온 상태(OF)
②SL 명령어가 R3 오퍼랜드를 읽기 직전(ID)
③SUB 명령어는 인출 완료 상태(IF)
→ 첫 명령어의 연산결과가 아직 R3로 저장(IS)되지 않아 2번째 명령어의 OF (R3 값 읽기)는 진행될 수 없어 스톨이 발생

ADD 명령어가 R3의 결과 저장을 완료했다(IS)
→파이프라인이 다시 정상적으로 동작

2.3 명령어 파이프라이닝

p82~p91

■ 분기 발생(명령어 종속성)에 의한 성능 저하의 최소화 방법

✓ 현대 CPU에서는 '분기 예측' + 'BTB'가 주류이고,'루프 버퍼'는 임베디드/저전력 CPU에서, '지연 분기'는 RISC 초창기 설계에서 주로

기법	동작 방식	장점	단점 / 한계
분기 예측 (Branch Prediction)	분기 결과(실행될지/안 될지)를 미리 **예측**하여 한쪽의 명령어를 가져옴 동적예측/정적예측	- 높은 성능 가능(특히 동적예측 시) - 최근 CPU에서 주류 기법	- 오예측 시 파이프라인 플러시 발생 → 성능 저하 - 예측 정확도에 따라 효과가 달라짐
분기 목적지 선인출 (Branch Target Prefetch / Branch Target Buffer, BTB)	분기 명령어의 목적지 명령을 미리 **고속 버퍼**에 저장해두고, 분기가 일어나면 즉시 버퍼의 명령어를 사용	- 예측이 맞으면 지연 최소화 - BTB로 빠른 분기 처리 가능	- BTB 저장공간 필요 - 방향 예측이 틀리면 여전히 플러시 발생
루프 버퍼 (Loop Buffer)	작은 루프의 명령어들을 **전용 버퍼**에 저장해 두어, 매번 메모리/캐시에서 가져오지 않고 재사용	- 루프 반복 시 페치 지연 제거 - 소비 전력 감소	- 작은 루프에만 효과적 - 루프 외부 코드에는 적용 불가
지연 분기 (Delayed Branch)	분기 명령어 바로 다음 명령어 1개 이상은 항상 실행되도록 규칙을 정함 (컴파일러가 빈 슬롯을 **유용한 명령으로 채움**)	- 하드웨어 복잡성 감소 - 파이프라인 지연 완화	- 유용한 명령어로 슬롯 채우기 어려운 경우 NOP 삽입 필요 → 성능 손해 - 최신 CPU에서는 잘 쓰이지 않음

플러시(Flush): 잘못 인출·실행된 명령어들을 파이프라인에서 모두 폐기하는 것

2.3 명령어 파이프라이닝

■ 분기 예측(branch prediction)

✓ 조건 분기를 만날 때마다 시스템은 분기할 것인지 여부를 미리 예측하는 것.

➢ 예측된 순서로 명령어를 진행하다가 만약 반대일 경우 버리는 방법

✓ 다음과 같은 두가지 방법이 있다

1) 정적분기 예측(Static Branch Prediction)

❖ 분기가 발생할 것인지 아닌지 미리 사전에 예측하고 실행 중에는 이를 변경하지 않는 것

❖ 컴파일 시에 분기 여부를 미리 결정

2) 동적분기 예측(Dynamic Branch Prediction)

❖ 동적분기 예측은 컴파일 할 때 예측하는 정적분기 예측과 달리 프로그램 실행도중에 분기를 예측하는 방법

❖ 분기 가능성 여부는 이전의 분기 기록(branch history table)을 보고 결정한다. (ARM11)

2.3 명령어 파이프라이닝

■ 지연 분기(Delay Branch) – 1/2

- ✓ 만일 조건 분기 명령어 앞에 위치한 명령어들의 그 실행 결과가 다른 명령어나 분기 조건에 영향을 주지 않는다면, 분기 명령어와 그 명령어를 순서를 바꾸어 실행하는 방법 → 컴파일러에서 지원
- ✓ 순서를 바꾸어 실행되는 명령어 구간이 분기 지연슬롯 (delay branch slot)
- ✓ 예제

```
Again:  LOAD 5, R1        ; R1 = 5
        SUB R2            ; R2 = R2 - 1
        BNN Again         ; Branch to Again if Not Negative
        ADD R4, R5, R3    ; R3 = R4 + R5


Again:  SUB R2            ; R2 = R2 - 1
        LOAD 5, R1        ; R1 = 5
        BNN Again         ;
        ADD R4, R5, R3    ; R3 = R4 + R5
```

연산이 끝나야 분기여부를 알 수 있다

분기결과에 영향을 주지 않는 명령어

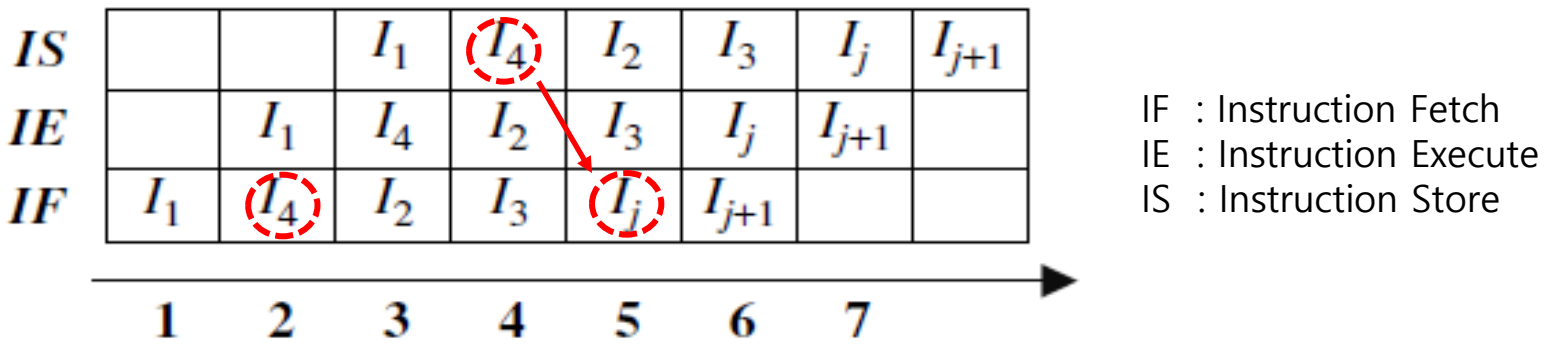
여기서 미리 연산을 실시해서 분기와 관련된 스톨이 없이 연속동작 하게 한다

분기 지연슬롯 이라고 한다.

2.3 명령어 파이프라이닝

■ 지연 분기(Delay Branch) – 2/2

- ✓ 명령어들의 순서를 재배치 → 컴파일러에서 재배치를 지원
- ✓ 예제
  - ➢ I_1, I_2, I_3, I_4 ,,, I_j, I_{j+1}이며 3단계 파이프라인(IF, IE, IS)
  - ➢ 명령어 코드에서 I_4 가 무조건 분기 명령이고, 목적 명령어 I_j
- ✓ 적용 방법
  - ➢ I_1, **I_4**, I_2, I_3, ,,, I_j, I_{j+1} 처럼 명령어의 순서를 바꾸어 재배치
  - ➢ I_2, I_3 명령어에 이어서 목적지의 명령어인 I_j가 바로 이어서 진행 된다.



IF : Instruction Fetch
IE : Instruction Execute
IS : Instruction Store

2.3 명령어 파이프라이닝

p82~p91

■ CPU 상태 레지스터(status register)



✓ 조건분기 명령어가 사용할 조건 플래그(condition flag)들 저장

✓ 조건 플래그의 종류

- 부호(S) 플래그 : 직전에 수행된 산술연산 결과값의 부호 비트를 저장
- 영(Z) 플래그 : 연산 결과값이 0 이면, 1
- 올림수(C) 플래그 : 덧셈이나 뺄셈에서 올림수(carry)나 빌림수(borrow)가 발생한 경우에 1로 세트
- 동등(E) 플래그 : 두 수를 비교한 결과가 같게 나왔을 경우에 1로 세트
- 오버플로우(V) 플래그 : 산술 연산 과정에서 오버플로우가 발생한 경우에 1로 세트

✓ 제어 플래그의 종류

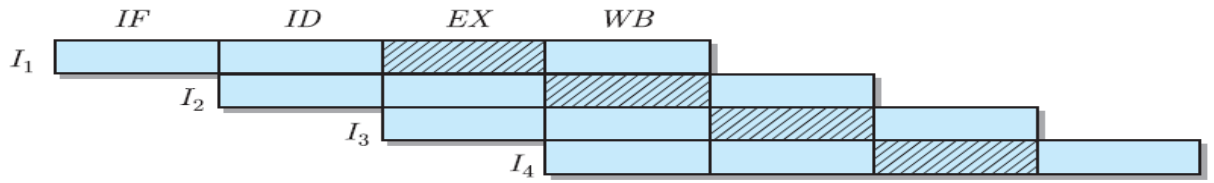
- 인터럽트(I) 플래그
  - 인터럽트 가능(interrupt enabled) 상태이면 0로 세트
  - 인터럽트 불가능(interrupt disabled) 상태이면 1로 세트
- 슈퍼바이저(P) 플래그
  - CPU의 실행 모드가 슈퍼바이저 모드(supervisor mode)이면 1,사용자 모드(user mode)면 0을 설정

2.3 명령어 파이프라이닝

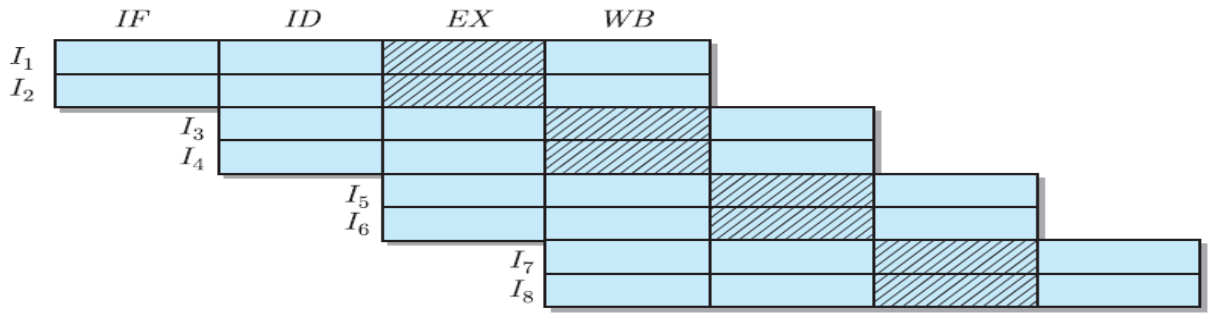
p91~p97

■ 슈퍼스칼라(Superscalar)

- ✓ CPU의 처리 속도를 더욱 높이기 위하여 내부에 복수 개의 명령어 파이프라인들을 포함시킨 구조
- ✓ 매 클록 주기마다 각 명령어 파이프라인이 별도의 명령어를 인출하여 동시에 실행할 수 있기 때문에,이론적으로는 프로그램 처리 속도가 파이프라인의 수만큼 향상 가능
- ✓ 파이프라인의 수 = m → m-way 슈퍼스칼라



(a) 일반적인 파이프라인의 명령어 실행 시간도



(b) 2-way 슈퍼스칼라의 명령어 실행 시간도

2.3 명령어 파이프라이닝

p91~p97

■ 슈퍼스칼라에 의한 속도향상(speedup: *Sp*)

✓ 단일 파이프라인에 의한 실행 시간 (*N*: 실행할 명령어 수)

$$T(1) = k + N - 1$$

✓ m-way 슈퍼스칼라에 의한 실행 시간

$$T(m) = k + \frac{N - m}{m}$$

✓ 속도 향상

$$Sp = \frac{T(1)}{T(m)} = \frac{k + N - 1}{k + (N - m)/m} = \frac{m(k + N - 1)}{N + m(k - 1)}$$

✓ 명령어 수 $N \rightarrow \infty$, $Sp \rightarrow m$

2.3 명령어 파이프라이닝

p91~p97

■ 슈퍼스칼라의 속도 저하 (Sp < m) 요인

✓ 저하 요인

- 파이프라인을 복수로 사용하는 기법이기 때문에 단일 파이프라인 사용시 보다 더욱 문제가 발생할 가능성이 높다
- 명령어들 간의 데이터 의존 관계
- 하드웨어(ALU, 레지스터, 등) 이용에 대한 경합 발생
  - 동시 실행 가능한 명령어 수 < m

✓ 해결책

- 명령어 실행 순서 재배치
  - 명령어들 간의 데이터 의존성 제거
- 하드웨어 추가(중복) 설치
  - 하드웨어(ALU, 레지스터, 등)에 대한 경합 감소

2.3 명령어 파이프라이닝

p91~p97

■ 동적 실행

✓ 명령어-수준 병렬성(instruction-level parallelism)

- ➢ 데이터 의존성이 없는 명령어들을 동시에 실행하는 것

✓ 동적 실행(dynamic execution)

- ➢ 파이프라이닝 및 슈퍼스칼라 구조에 추가적으로 도입된 설계 개념 → **동적 분기 예측**
- ➢ 명령어-수준 병렬성을 유지하기 위한 기법
- ➢ 프로그램을 컴파일하여 생성한 실행 코드 파일에 나열되어 있는 명령어들을 원래의 순서와는 다르게 변경하면서 실행하는 기법
- ➢ 원래의 순서대로 명령어를 실행할 때 발생되는 데이터 의존성으로 인한 슈퍼스칼라 파이프라인의 유휴 사이클(idle cycle) 감축

✓ 동적 명령어 스케줄링(dynamic instruction scheduling)

- ➢ 하드웨어 자원들을 최대한 효율적으로 이용할 수 있도록 실행시간(run-time) 중에 명령어 순서를 재배열해 주어 동적 실행을 지원해주는 기법

2.3 명령어 파이프라이닝

p91~p97

■ 동적 실행 순서 및 구조

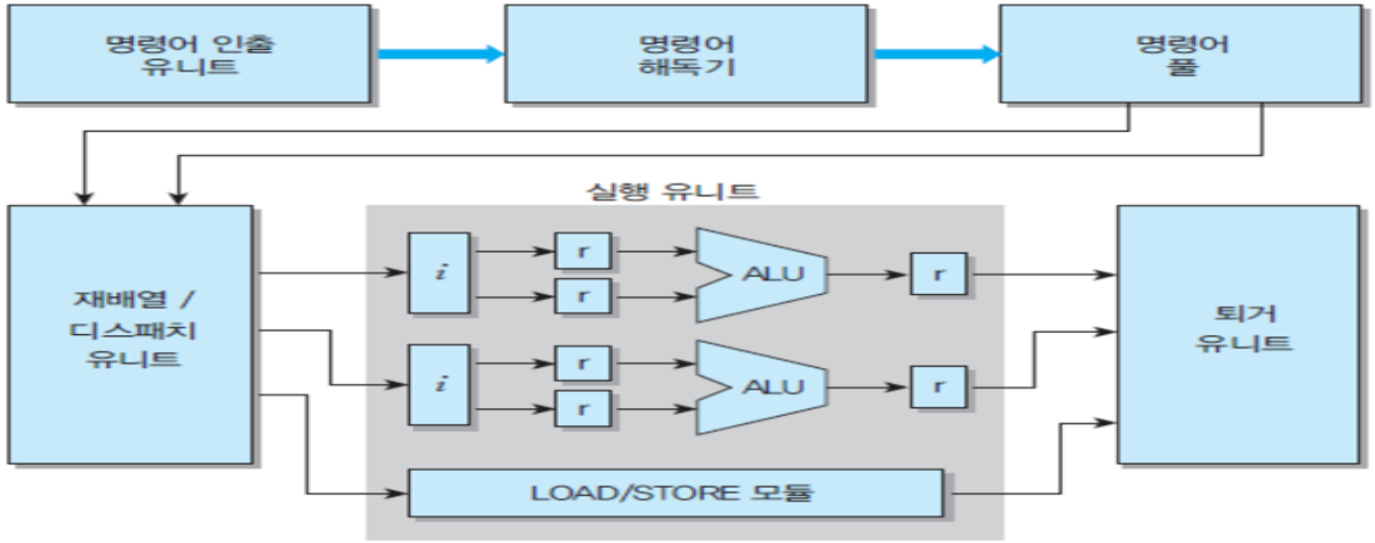
① 기억장치로부터 명령어들을 순서대로 인출

② 명령어를 해독하고, 그 결과를 명령어 풀에 저장

③ 원래 순서상 연속적인 명령어들이 데이터 의존성으로 인해 동시 처리 불가능한 경우에는 명령어 풀에서 그 다음에 위치한 명령어들을 검색하여 의존성이 존재하지 않는 명령어를 찾아냄

④ 동시처리 가능한 명령어들의 연산을 수행할 연산장치들(예: ALU, 기억장치 액세스 유니트)을 배정하고, 명령어와 데이터를 그 장치들로 디스패치 시켜 처리

⑤ 퇴거 유니트에서 연산처리 결과를 저장하고, 명령어를 제거함으로써 실행을 종료



2.3 명령어 파이프라이닝

p91~p97

■ 동적 실행의 예(2-way 슈퍼스칼라)

✓ 명령어 순서

1) I1 : LOAD R1, X ; R1 ← M[X]
2) I2 : ADD R1, R2 ; R1 ← R1 + R2
3) I3 : SUB R3, R4 ; R3 ← R3 - R4
4) I4 : STOR Y, R3 ; M[Y] ← R3

✓ 실행 순서

1) 매 사이클마다 두 명령어씩(I1과 I2, I3과 I4)을 순서대로 인출하여 해독하고, 명령어 풀에 저장
2) 재배열/디스패치 유니트가 동시 실행 가능한 명령어들 검색
   - ❖ I1과 I2 간 데이터 의존성 존재 : 동시 실행 불가
   - ❖ 원래 순서와 다르게, (I1과 I3) (I2와 I4)를 순차적으로 동시 실행
3) 퇴거 유니트는 결과를 저장하고, 실행 완료된 명령어들 제거

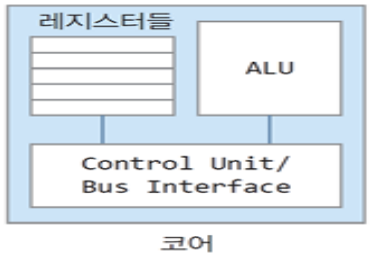
2.3 명령어 파이프라이닝

p97~p100

■ 멀티-코어 프로세서

✓ CPU 코어(core)

➢ 명령어 실행에 필요한 CPU 내부의 핵심 하드웨어 모듈



(a) 코어의 내부 구조

✓ 멀티-코어 프로세서(multi-core processor)

❖ 여러 개의 CPU 코어들을 하나의 CPU 칩에 포함시킨 프로세서

❖ 칩-레벨 다중프로세서(chip-level multiprocessor) 혹은 단일-칩 다중프로세서(multiprocessor-on-a-chip)이라고도 부름

➢ 듀얼-코어(dual-core)

❖ 하나의 CPU 내부에 두 개의 코어 포함

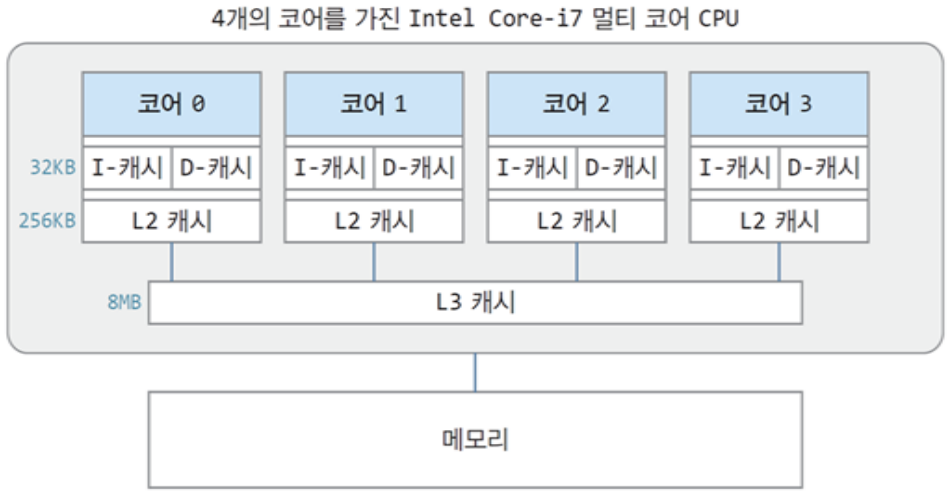
➢ 근래에는 더욱 많은 수의 코어를 가진 CPU 들이 등장

'멀티코어 기술'과 '명령어 파이프라이닝'은 **서로 다른 차원의 기술**인데 교재에서는 함께 설명하고 있음. 멀티코어 기술은 파이프라이닝을 구현는 방법이 아님.

- 명령어 파이프라이닝은 단일 코어(Single Core)에서의 효율을 극대화하는 기술
- 멀티코어 기술은 CPU 칩 하나에 여러 개의 코어(처리 장치)를 넣는 기술
- 멀티코어 CPU의 각 코어 내부에서도 파이프라이닝이 동작

→ 파이프라이닝 = 코어 안에서의 병렬성, 멀티코어 = 코어 단위 병렬성

- 병렬성(Parallelism)이란 여러 연산이 동시에 수행될 수 있는 성질 또는 구조



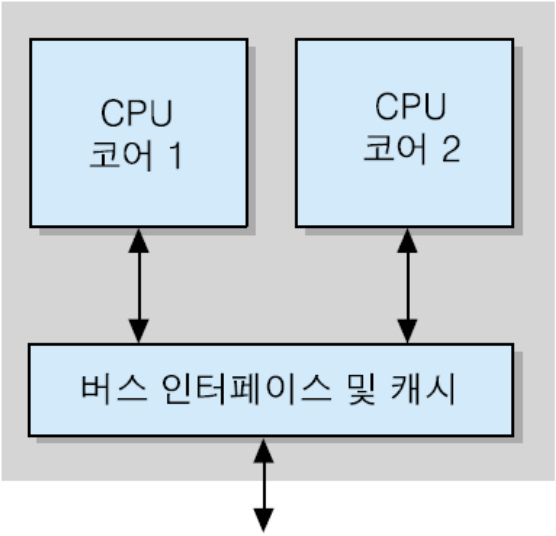
(b) 4개의 코어를 가진 Intel Core-i7 멀티 코어 CPU 사례

2.3 명령어 파이프라이닝

p97~p100

■ 듀얼-코어 프로세서

- ✓ 단일-코어 프로세서에 비하여 2배의 속도 향상 기대
- ✓ 코어들은 내부 캐시와 시스템 버스 인터페이스만 공유
- ✓ 코어 별로 독립적 프로그램 실행
  - ➢ 멀티-태스킹(multi-tasking) 혹은 멀티-스레딩(multi-threading) 지원



2.3 명령어 파이프라이닝

p97~p100

■ 멀티-스레딩

✓ 스레드(thread) = 쓰레드

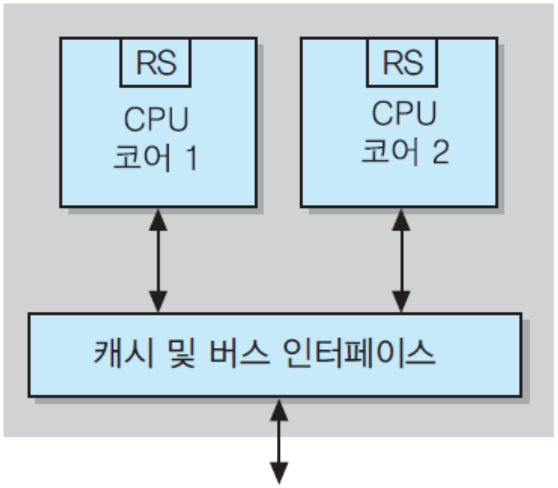
➢ 독립적으로 실행될 수 있는 최소 크기의 프로그램 단위

1) 단일-스레드 모델

➢ 각 코어별로 독립적으로 스레드를 한 개씩 처리하는 구조

➢ 처리 중인 스레드에 대한 시스템 상태, 데이터 및 주소 정보를 레지스터 세트(RS)에 저장

❖ RS(Register Set, 레지스터 세트) → PC, SP, 상태 레지스터, 데이터 레지스터, 주소 레지스터 등 모든 레지스터의 집합(Set)



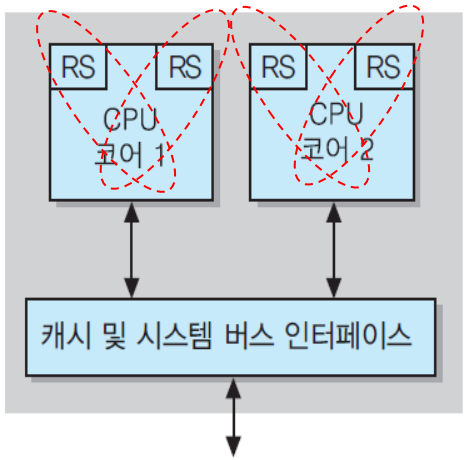
(a) 단일-스레드 모델

2.3 명령어 파이프라이닝

p97~p100

■ 멀티-스레드 모델

2) 멀티-스레드 모델 – 1/2



(b) 멀티-스레드 모델

- CPU(코어)의 유휴 자원을 활용해 처리 효율을 높이고, 멀티태스킹 성능을 향상시키는 기술
- 하나의 물리 코어에 **2개의 RS(레지스터 세트)**를 갖도록 설계하여,
- 운영체제 입장에서는 하나의 물리 코어 내부에 **2개의 코어**를 갖는 것 처럼 보이게 하는 기술
- 이러한 2개의 코어를 **논리 코어(논리 프로세서)**라고 하며 각 논리 코어들은 각각 별도의 스레드를 실행
- 2개의 논리 코어들은 물리 코어의 하드웨어 자원을 공유(파이프라인, 레지스터, 실행 유닛 등)
- 논리 코어는 동시에 동작되는 것이 아니라, 특정 순간에는 운영체제가 선택한 하나의 논리 코어만이 실행됨
- 인텔에서는 **하이퍼스레딩 기술**(Hyper-Threading, HT), AMD에서는 **SMT**(Simultaneous Multi-Threading)라고 부름
- 논리 코어가 실행 중에 다른 논리 코어로 변경될 때, 현재의 RS는 그대로 두고 다른 RS를 이용해 작업을 진행
  - RS가 하나인 경우 다른 스레드를 처리할 때 기존의 RS 값들을 주기억장치에 있는 스택에 백업한 후 다른 스레드를 처리하고 다시 원래의 스레드를 처리하기 위해 스택에 있는 RS 값들을 읽어와 복원하면서 처리 → 스택 작업에 많은 시간이 소요(스택 메모리는 CPU 외부의 주기억장치 저장 공간을 이용)
  - 레지스터의 정보를 스택 메모리가 아닌 CPU 내부의 RS에 저장해 두기 때문에 속도가 매우 빨라짐 → 실제 물리 코어를 2배로 늘리는 것 보다는 속도가 떨어지나 가격적인 측면에서 매우 유리

2.3 명령어 파이프라이닝

p97~p100

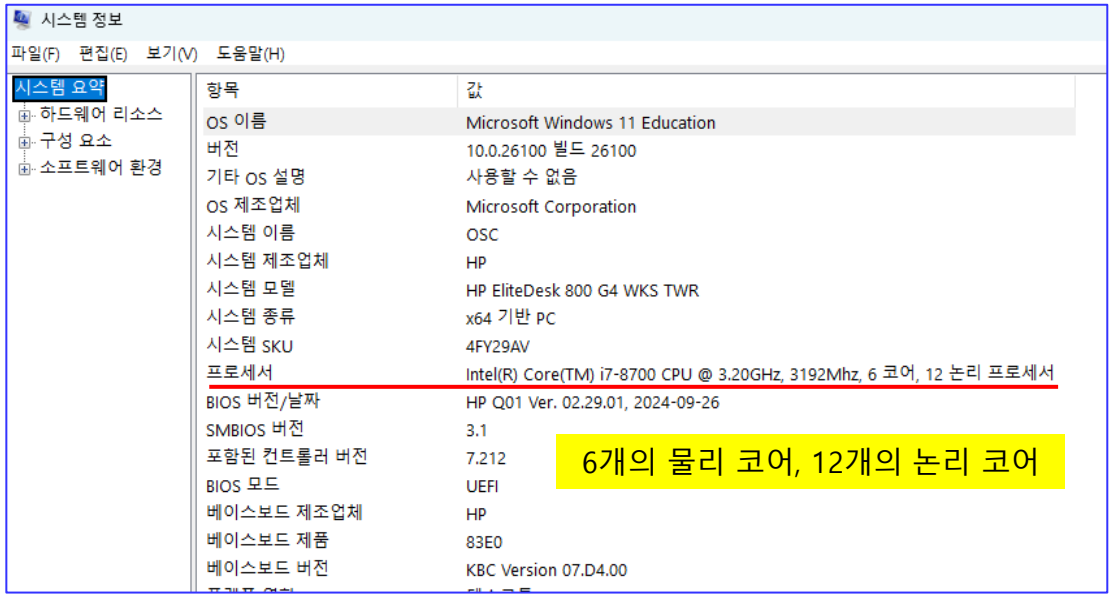
■ 멀티-스레드 모델

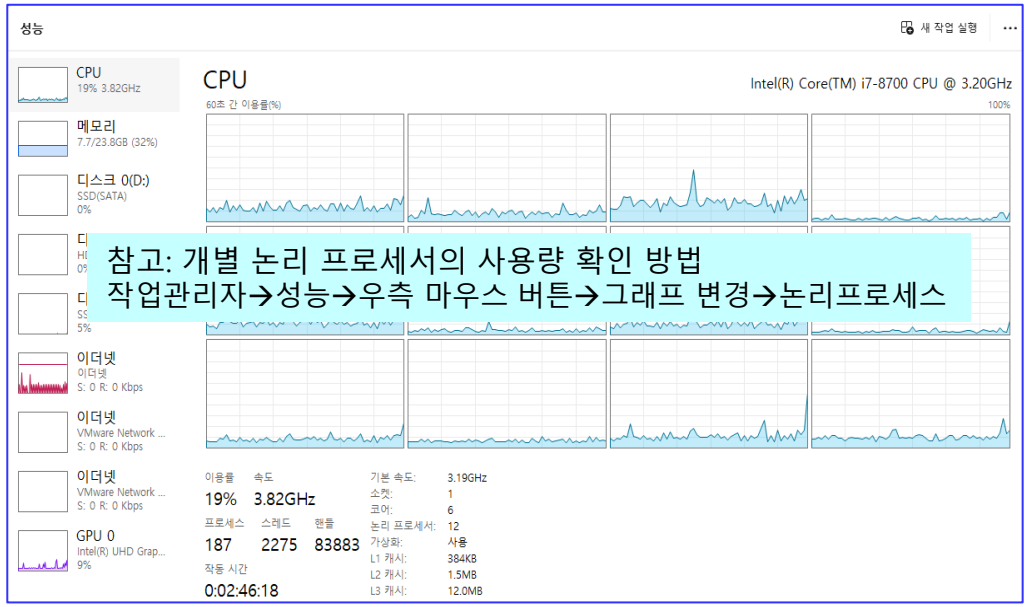
2) 멀티-스레드 모델 – 2/2

✓ 적용 사례

- ➢ Intel Core i7-9700K → HT 미지원 (8코어 8스레드)
- ➢ Intel Core i7-10700K → HT 지원 (8코어 16스레드)
- ➢ AMD Ryzen 7 3700X → SMT 지원 (8코어 16스레드)

✓ 윈도우 검색창에 'msinfo32'를 입력하면 본인 CPU의 코어, 논리 프로세스 수를 알 수 있음





2.3 명령어 파이프라이닝

p97~p100

■ 혼합형 멀티-코어 프로세서

- ✓ Hybrid architecture(하이브리드 아키텍처)
- ✓ big.LITTLE 구조라고도 부름
- ✓ **크기와 성능이 다른 CPU 코어**들을 한 칩에 포함
- ✓ 저전력 소모를 요구하는 휴대용 기기를 위한 멀티-코어 프로세서
- ✓ 단순한 태스크들과 복잡한 태스크들이 혼재된 컴퓨팅 환경에서 사용
- ✓ 인텔에서는 12세대 CPU 부터 적용
  - ➢ 복잡한 태스크는 고성능(일반적인) 코어가 처리 → P-코어
  - ➢ 단순한 태스크는 단순 구조의 저전력 코어가 처리 → E-코어

■ [추가] ARM 계열의 혼합형 멀티-코어 프로세서 구조

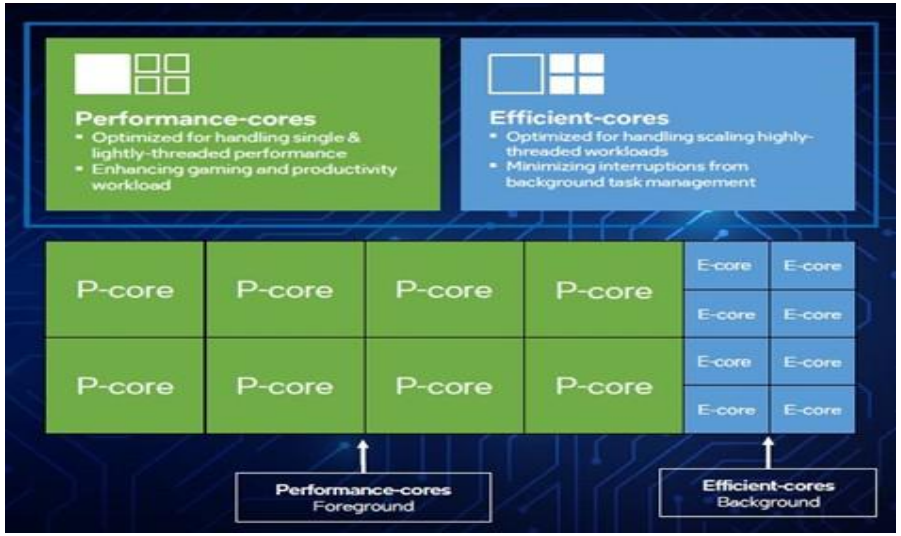
- ✓ 전원의 효율성을 위해 Little, Big 코어를 복수 개 사용
- ✓ 연산의 양에 따라 동작되는 코어의 개수를 늘려가는 방식을 채택해 전원 사용의 최소화



< 삼성전자 엑시노드 AP >

2.3 명령어 파이프라이닝

■ Intel사의 혼합형 멀티-코어 프로세서 구조



✓ P-코어(performance-core)

- ➢ 고성능 CPU 코어, 복잡한 작업 처리
- ➢ 멀티-스레딩(하이퍼스레딩) 지원
  - ❖ 주의: 15세대 (2024) 일부 CPU에서는 멀티-스레딩 미지원

✓ E-코어(efficient-core)

- ➢ 단순 구조, 저속, 저전력 CPU 코어
- ➢ 작은 태스크나 백그라운드 작업 처리
- ➢ 멀티-스레딩(하이퍼스레딩) 미지원

✓ 예: Intel Core i9-12900K CPU (P: 8C16T, E:8C/8T)

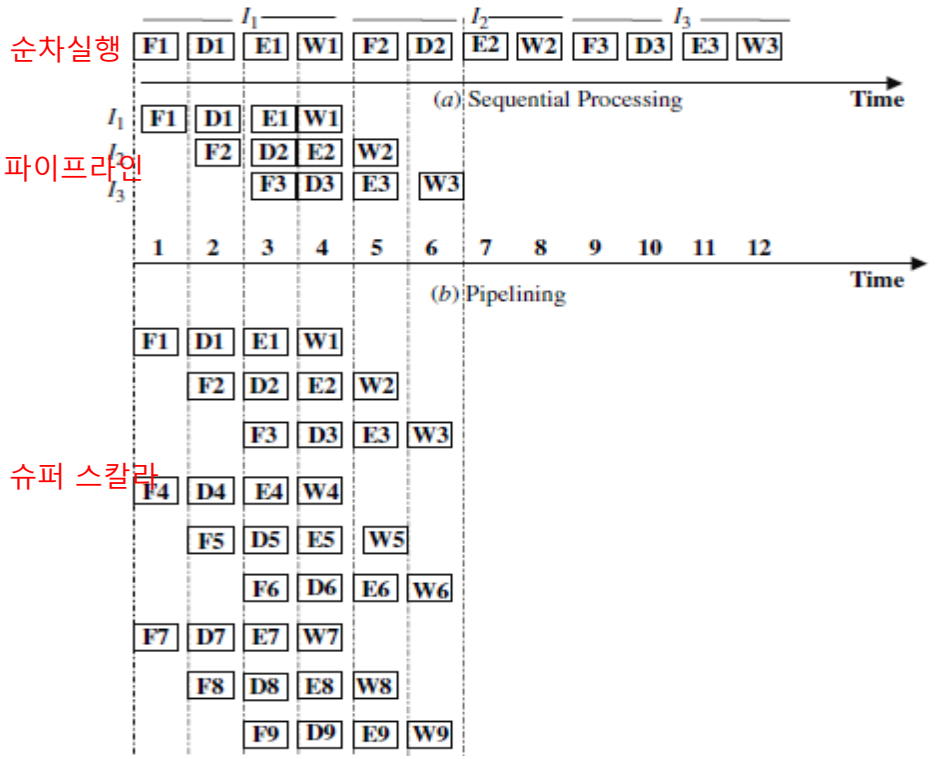
- ➢ 8개의 물리 P-코어, 16개의 논리 프로세서(16개의 스레드 처리)
- ➢ 8개의 물리 E-코어(8개의 스레드 처리)

세대	대표 CPU	P코어	E코어	총 코어 수	스레드 수	비고
11세대 (Rocket Lake, 2021)	i9-11900K	8	0	8	16	하이브리드 아키텍처 **미적용**, 전통적 HT
	i7-11700K	8	0	8	16	
	i5-11600K	6	0	6	12	
12세대 (Alder Lake, 2021)	i9-12900K	8	8	16	24	P/E 코어 **처음 도입**
	i7-12700K	8	4	12	20	
	i5-12600K	6	4	10	16	
13세대 (Raptor Lake, 2022)	i9-13900K	8	16	24	32	E코어 대폭 증가
	i7-13700K	8	8	16	24	
	i5-13600K	6	8	14	20	
14세대 (Raptor Lake Refresh, 2023)	i9-14900K	8	16	24	32	구조는 13세대와 동일, 클록↑
	i7-14700K	8	12	20	28	
	i5-14600K	6	8	14	20	

2.3 명령어 파이프라이닝

■ [추가] 명령어 수준 병렬성(Instruction-Level Parallelism: ILP)

- ✓ **하나의 코어**에서 동시에 수행되어도 서로 영향을 주지 않는 명령어들의 관계를 **명령어 수준 병렬성(ILP)**라고 한다
- ✓ ILP를 구현하기 위해서는 명령어/데이터 종속성을 분석하고 스케줄링 해야 함
- ✓ 참고: 병렬 처리(Parallel Processing)
  - ➢ 하나의 큰 문제나 작업을 여러 개의 작은 작업 단위로 나누고, 이를 여러 처리 장치(코어, CPU, GPU, 노드)가 동시에 수행하는 것
  - ➢ 하드웨어적 구현 방법은 **단일 코어 내의 ILP**, GPU 기반 병렬 처리, 멀티코어/멀티프로세서 시스템 등
- ✓ 참고: 병렬성(Parallelism)
  - ➢ 여러 연산이 동시에 수행될 수 있는 성질 또는 구조
  - ➢ 병렬 처리는 실제 구현 방법이며 병렬성은 그 가능성과 특성을 나타내는 추상적인 개념

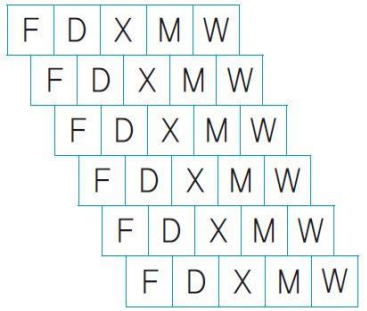


2.3 명령어 파이프라이닝

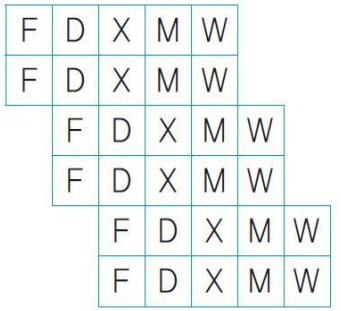
■ [추가] 명령어 수준 병렬성 구조의 종류

1) 슈퍼파이프라이닝 (Superpipelining)
   - ➢ 기존의 파이프라인 단계를 더욱 잘게 쪼개어 파이프라인의 깊이(depth)를 늘리는 기술
   - ➢ 파이프라이닝과 동일한 의미로 생각해도 됨
2) 슈퍼스칼라 (Superscalar)
   - ➢ 병렬의 복수개의 파이프라인 구조 → Cost 높다
3) VLIW(Very Long Instruction Word)
   - ➢ 긴 길이의 명령어 하나가 내부에서 여러가지 연산을 동시에 실시하는 구조
     - ❖ 인출과 해독은 하나의 회로에 의해 실시하기 때문에 Cost가 낮다
   - ➢ 컴파일러가 컴파일 시에 H/W에 맞게끔 정적으로 분석하고 스케줄링을 하여 명령어 그룹을 조합

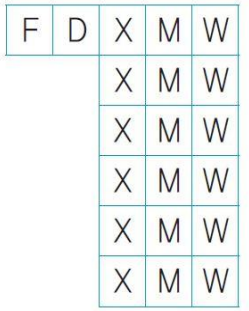
근래의 대부분의 CPU 구조는 슈퍼파이프라이닝+ 슈퍼스칼라



(a) 슈퍼파이프라이닝



(b) 슈퍼스칼라 파이프라이닝



(c) VLIW

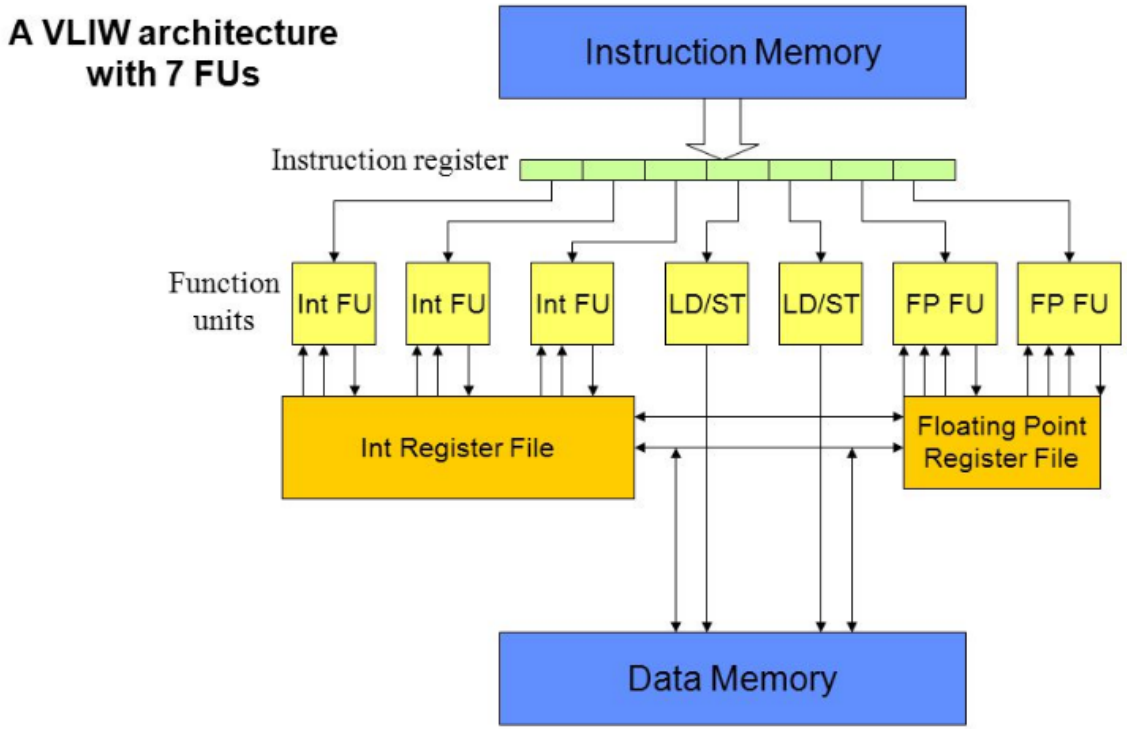
구분	슈퍼파이프라이닝	슈퍼스칼라	VLIW
핵심 개념	파이프라인을 더 깊게 (Deeper)	파이프라인을 더 넓게(Wider)	컴파일러가 짠 명령어 묶음 (Bundle)
병렬 처리 주체	하드웨어 (CPU)	하드웨어 (CPU)	소프트웨어 (컴파일러)
작동 원리	파이프라인 단계를 잘게 쪼개 클록 속도 향상	여러 개의 파이프라인을 두어 동시에 여러 명령어 실행	컴파일러가 병렬 실행 가능한 연산들을 미리 하나의 긴 명령어로 묶음
하드웨어 복잡도	중간	높음 (실시간 의존성 분석 회로 필요)	낮음 (단순히 계획대로 실행)
컴파일러 의존도	낮음	중간	매우 높음 (성능을 컴파일러가 결정)
비유	더 길고 세분화된 조립 라인	여러 개의 조립 라인을 동시에 가동	본사에서 완벽히 계획해 보낸 밀키트

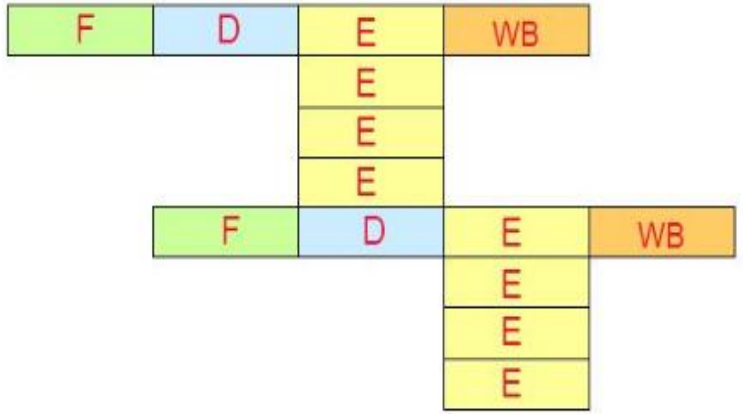
2.3 명령어 파이프라이닝

■ [추가] 명령어 수준 병렬성 구조의 종류

- ✓ VLIW(Very Long Instruction Word) – 계속
  - ➢ DSP와 GPU(초창기)에 주로 사용
  - ➢ 저가의 H/W로도 높은 성능을 얻지만 컴파일러는 복잡해짐

근래의 GPU들은 지나치게 컴파일러 의존적인 VLIW 방식을 사용하지 않고 SIMD 구조를 사용→ 8장 학습





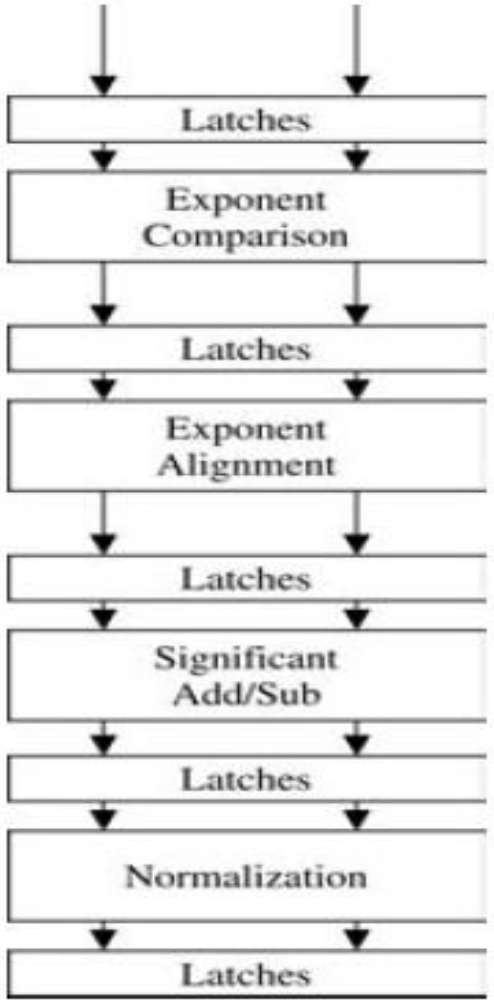
명령어 코드가 길지만 하나로 취급되기 때문에 인출과 해독은 하나의 회로에 의해 이루어지고, 각 연산의 실행 사이클만 여러 개의 유니트(ALU)들로 나누어져 동시에 처리된다는 점

2.3 명령어 파이프라이닝

■ [추가] 산술 파이프라인

✓ 부동소수점 산술 파이프라인

- 부동소수점 수의 연산에서 덧셈과 뺄셈은 지수 조정 단계를 지수부와 가수부에 대해 별도로 처리하기 위해 모두 4단계로 처리된다.
- 부동소수점에서도 파이프라인이 가능하며 부동소수점의 가산기는 다음과 같은 4단계 파이프라인을 갖는다
  1. 지수 비교(EC)
  2. 지수 조정(EA)
  3. 덧셈(AD)
  4. 정규화(NZ)



Thank you